Auto-EAD: An Automated Ensemble Anomaly Detection Framework for the CEN-IoT

Lei Ding, Wenli Shang, Shuang Wang, *Member, IEEE*, Haoran Gao, and Lianhai Wang, *Member, IEEE*

***Abstract*— The Internet of Things (IoT) based on consumer electronic networks (CEN-IoT) is confronting diverse, stealthy, and large-scale network attacks. The targets of attackers are no longer limited to individual devices or data, but also extend to multiple domains, including device hijacking, data exfiltration, and service disruption. Anomaly detection based on machine learning, as an effective defensive measure, is gradually being applied to IoT systems to enhance cybersecurity. Although anomaly detection algorithms based on artificial intelligence/machine learning are widely employed in constructing cybersecurity mechanisms, such technologies not only require substantial support from human experts but also suffer from model drift, posing considerable challenges to the development of autonomous cybersecurity measures. This paper proposes an Automated Ensemble Anomaly Detection Framework (Auto-EAD) for the automatic identification of abnormal behaviors in dynamic consumer electronics network environments. The proposed model consists of automated data preprocessing, automated feature engineering, automated base model learning with hyperparameter optimization, automated model selection, and automated model integration modules. Specifically, the primary automated data preprocessing techniques employed include automatic data transformation, inverse transformation, feature standardization, and data balancing. The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) is utilized for hyperparameter optimization (HPO) of tree-based machine learning (ML) models, while a novel stair ensemble method is proposed for automated model ensemble.This model was developed to minimize human intervention and achieve full-lifecycle automation of anomaly detection. The Auto-EAD framework was evaluated on three public datasets (SWaT, WADI, and IoT Network Intrusion Dataset), with results showing superior performance of the proposed model compared to state-of-the-art methods, and thereby highlighting its effectiveness in handling and detecting abnormal behaviors in dynamically complex network environments. The present paper offers a novel approach to the construction of cybersecurity ecosystems with autonomous decision-making and response capabilities.**

***Index Terms*— Anomaly detection, automated machine learning, ensemble model, consumer electronics network.**

Received 16 September 2025; revised 12 November 2025 and 15 December 2025; accepted 1 January 2026. Date of publication 12 January 2026; date of current version 25 March 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62173101; in part by the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant 2024ZD018; and in part by the Fundamental Research Funds for the Central Universities under Grant 3122025054. *(Corresponding author: Wenli Shang.)*

Lei Ding and Haoran Gao are with the School of Cyber Science and Technology, Guangzhou University, Guangzhou 510006, China (e-mail: 1112206007@e.gzhu.edu.cn; ghr36369@e.gzhu.edu.cn).

Wenli Shang is with the School of Electronics and Communication Engineering, Guangzhou University, Guangzhou 510006, China (e-mail: shangwl@gzhu.edu.cn).

Shuang Wang is with the Institute of Science, Technology and Innovation, Civil Aviation University of China, Tianjin 300300, China (e-mail: s-wang@cauc.edu.cn).

Lianhai Wang is with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250353, China (e-mail: wanglh@sdas.org).

Digital Object Identifier 10.1109/TCE.2026.3651524

I. Introduction

THE Internet of Things (IoT) based on consumer electronic networks (CEN-IoT) [1]takes smartphones, smart home appliances, and remote-control devices as core terminals, achieves interconnection via network technologies including Wi-Fi, Bluetooth, and NB-IoT, enables data processing and intelligent control with the support of cloud platforms and edge computing, and adopts standardized protocols such as MQTT to ensure cross-device compatibility and secure data transmission [2]. Its applications span service scenarios including smart home linkage, remote health monitoring, smart manufacturing facilities, and intelligent transportation [3]. For instance, in a smart home scenario, when users enable the 'away-from-home mode' via a mobile application, all indoor lighting, power sockets, and air conditioning systems will automatically shut down, smart door locks will engage, and security cameras will switch to an armed state [4].

In intelligent manufacturin scenarios, the CEN-IoT system adopts a four-layer architecture (terminal layer, network layer, platform layer, and application layer). The terminal layer incorporates production equipment and intelligent handheld terminals, while the network layer leverages 5G and industrial Ethernet to achieve reliable data transmission. The platform layer conducts data processing through an industrial cloud platform, and the application layer offers services such as equipment health management. Embedded sensors gather real-time operational data, which is transmitted to the cloud platform. Advanced algorithms are then employed for fault analysis and prediction, thereby facilitating early warnings. Furthermore, multi-device collaborative operation is achieved. In practical applications, this model remarkably reduces equipment failure rates, improves core production efficiency, strengthens quality traceability capabilities, and comprehensively upgrades the overall production efficiency and quality control standards [5], [6], [7], [8].

With the in-depth integration of the CEN-IoT and the Internet, CEN-IoT is increasingly vulnerable to network attacks [9], [10]. The primary causes are threefold: (1) Most CEN-IoT devices inherently suffer from vulnerabilities including the absence of hardware security modules, inadequate firmware update mechanisms, and default weak passwords [11]; (2) Within network and communication links, design flaws

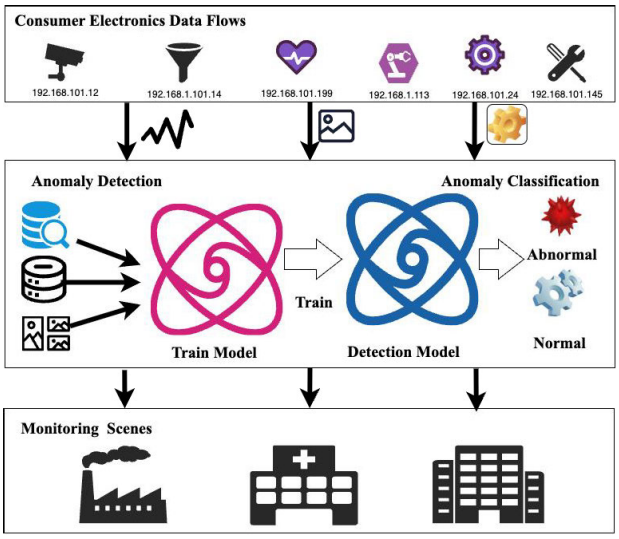

Fig. 1. Workflow of network anomaly detection for consumer electronic devices.

exist in plaintext data transmission, weakly encrypted protocols (e.g., HTTP, Telnet), and IoT-specific protocols (e.g., MQTT, Zigbee)—these vulnerabilities render CEN-IoT systems susceptible to network attacks, including eavesdropping, data tampering, and lateral movement [12], [13]; (3) Cloud platforms supporting CEN-IoT systems exhibit vulnerabilities such as SQL injection and unauthorized access, coupled with the absence of API authentication mechanisms and inadequate protection of user privacy data—these issues expose CEN-IoT devices to risks of batch hijacking or sensitive data leakage. In summary, the current network attacks targeting CEN-IoT systems exhibit three key characteristics: diversity, concealment, and large scale [14]. Attackers' targets are no longer limited to individual devices or isolated data, but extend to multiple dimensions, including device hijacking, data theft, and service disruption.

In CEN-IoT scenarios, smart home devices face various attacks: exploiting smart camera vulnerabilities for unauthorized monitoring, hijacking smart socket control to cause malicious power outages, tampering with smart refrigerator energy consumption data, or deploying automated scripts through universal smart camera vulnerabilities to compromise hundreds of thousands or even millions of devices within hours. These compromised devices form large-scale botnets, used to launch distributed denial-of-service (DDoS) attacks or engage in illegal activities such as cryptocurrency mining [15]. Traditional passive defense mechanisms can no longer tackle such sophisticated attack vectors. Therefore, active defense mechanisms centered on anomaly detection have emerged as a critical enabler for safeguarding CEN-IoT system security [16]. Fig. 1 showsthe workflow of network anomaly detection for CEN-IoT.

However, three persistent challenges hinder CEN-IoT broad application [10]. (1) Traditional anomaly detection methods primarily involve processes such as manual data annotation and the setting of detection thresholds. However, these processes demand extensive manual intervention, heavily depend on the professional expertise of personnel, exhibit low learning efficiency, and fail to adequately meet the real-time requirements of CEN-IoT systems. (2) The number of hyperparameters in anomaly detection models typically ranges from 10 to 20. The optimization space formed by these hyperparameters is extremely high-dimensional, and complex nonlinear correlations exist between hyperparameters—this renders hyperparameter optimization a challenging global optimization problem. Existing hyperparameter optimization methods impose a heavy computational burden and are frequently prone to trapping in local optima. (3) Traditional anomaly detection approaches are incapable of addressing the dynamic variations in CEN-IoT systems. Traditional single-model approaches or static integration strategies often exhibit significant performance disparities across different operating conditions, leading to substantial fluctuations in detection outcomes.

Automated ML (AutoML) autonomously derives optimal strategies by automating model selection, feature engineering, HPO, and model ensembling [17]. This approach has been widely applied across diverse domains [18]. For example, Li et al. [19] proposed an automated framework for constructing statistical learning-based anomaly detection models, which streamlines parameter optimization for non-neural network algorithms. AutoML-driven autonomous anomaly detection enables the real-time monitoring of system activities, the precise identification of abnormal patterns, and the swift localization of potential attack vectors. However, most existing AutoML-based anomaly detection methods are designed for general application scenarios. At the same time, anomaly detection of consumer electronics system is a complex task characterized by high real-time performance requirements, high stability requirements, and multiple operating conditions—factors that constrain the broad application of AutoML-driven methods.

Inspired by AutoML, we propose an automated ensemble anomaly detection model (Auto-EAD) for the CEN-IoT. Unlike traditional models that depend on human expertise, Auto-EAD integrates automated data mining and learning mechanisms, with an emphasis on efficient data stream processing, multi-model integration, and HPO to enhance the accuracy of anomaly detection.

The main contributions of this study are as follows:

(1) We developed an AutoML framework to enhance the tracking efficiency of model learning. It incorporates meta-learning, ensemble construction, and HPO. This enables the comprehensive automation of data processing, feature engineering, model selection, and parameter optimization in IoT anomaly detection. This framework notably boosts the tracking efficiency of model learning and minimizes reliance on manual expertise.

(2) We propose a method for optimizing the hyperparameters of the detection model using adaptive evolution. The covariance matrix is dynamically adjusted based on the 在 population distribution and historical information to capture the correlations among the model hyperparameters. The search direction of the hyperparameters is then updated without any gradient calculation. The sampling of multiple hyperparameter

combinations and comprehensive evaluations smooth out the fluctuations in the objective function, accelerating convergence to the global optimum.

(3) The proposed Stair ensemble approach gradually adds models to enhance learning performance and mitigate overfitting. Through a multilevel fusion mechanism, the meta-models mutually correct their learning errors, improving the stability of the integrated model. Through the dynamic adjustment of the meta-model weights, the integrated model maintains high performance and improved interoperability.

The remainder of this paper is structured as follows: Section II reviews anomaly detection methods. Section III details the proposed Auto-EAD framework. Section IV outlines the experimental design, and Section V presents and discusses the results. Finally, Section VI summarizes the paper and suggests future research directions.

II. Related Work

A. Anomaly Detection Technique

Anomaly detection techniques can be broadly classified into three categories: signature-based, statistics-based, and ML–based techniques.

Signature-based anomaly detection techniques accurately identify known attacks by extracting and matching characteristic patterns from network traffic or system behaviors. Bahri et al. [20] proposed an anomaly detection system (ADS) tailored to industrial protocols, improving detection accuracy by refining the signature rule-base through detailed analysis of Modbus RTU/ASCII vulnerabilities. Zheng et al. [21] proposed a hybrid Transformer-Convolutional Signature Network (T-CSN) architecture, which innovatively integrates the Transformer and Convolutional Neural Network (CNN) frameworks. This architecture can effectively capture subtle deformation features between genuine signatures and skilled forged signatures, thereby playing a critical role in enhancing the anomaly detection accuracy of offline signature authentication systems. He et al. [22] combined Variational Autoencoders (VAEs)—a type of generative model—with Sequence-to-Sequence (Seq2Seq) structures, and incorporated Graph Neural Networks (GNNs) to learn topological information. This approach provides an innovative spatiotemporal deep learning solution for signature-based anomaly detection in cloud systems, with network traffic signatures as a typical application scenari. Signature-based anomaly detection techniques enable the precise identification of known attacks through the extraction and matching of distinctive feature patterns. Furthermore, these methods exhibit robust adaptability to domain-specific scenarios—such as industrial communication protocols and cloud computing environments—with detection performance further enhanced via the integration of multi-neural network frameworks, refinement of signature rule bases, and other advanced optimization strategies. Nevertheless, a critical limitation persists in their inadequate capability to effectively detect unknown or zero-day attacks.

Statistics-based anomaly detection techniques exhibit robust performance on relatively stable time series or normally distributed data. However, they often falter in accurately identifying anomalies in data with substantial variations in statistical characteristics or in cases requiring joint assessment of multiple variables, necessitating the use of more sophisticated approaches for achieving reliable anomaly detection in such scenarios. Xiong et al. [23] proposed a spatial information-enhanced Transformer (SI-Transformer) model, which integrates Graph Neural Networks (GNNs) into the original Anomaly Transformer (A-Transformer) framework. This model generates high-dimensional correlated difference features to more effectively capture spatiotemporal dependencies in multivariate time series (MTS) data, thereby enabling more robust anomaly detection. Sukhostat et al. [24] introduced an anomaly detection technique relying on statistical features and applied it to anomaly detection in in-vehicle CAN networks. Statistics-based anomaly detection techniques conduct anomaly identification exclusively relying on a priori knowledge of data or their inherent statistical characteristics. While demonstrating efficacy for relatively stable time series or data sequences adhering to a normal distribution, these approaches fail to achieve precise anomaly detection for data with significant fluctuations in statistical properties or those that necessitate the joint assessment of multiple variables. Consequently, there is an imperative need for more robust detection paradigms to address such complex scenarios.

ML-based anomaly detection techniques identify abnormal behaviors by constructing data-driven models to learn normal behavioral patterns from historical data and then recognizing deviations from these patterns. Bai et al. [25] proposed an anomaly detection method based on the Parallel Attention Transformer (PA-Transformer), which leverages the Transformer architecture to capture complex temporal dependencies in time series data, thereby enabling effective anomaly detection. Gu et al. [26] proposed a novel industrial anomaly detection method based on Large Visual Language Models (LVLMs)—a class of advanced multimodal models—which eliminates the need for manual threshold tuning and can directly assess both the existence and location of anomalies in industrial scenarios. Wu et al. [27] proposed an interpretable and efficient deep-learning framework for anomaly detection, reporting higher training speed and effectiveness compared with current methods. Recurrent neural networks, such as long short-term memory (LSTM) [28] and gated recurrent unit [29], along with attention-based models [30], capture contextual relationships in time-series data. These models learn normal patterns by capturing temporal dependencies, thus identifying anomalies in time series. ML-based anomaly detection techniques can learn inherent normal behavioral patterns through data-driven paradigms, capture intricate dependencies and contextual correlations within time series data, and exhibit prominent advantages including the elimination of manual parameter tuning, high training efficiency, favorable interpretability, and robust adaptability to industrial application scenarios. Nevertheless, these approaches inherently suffer from non-negligible limitations: strong reliance on data quality and quantity, considerable challenges in hyperparameter optimization (HPO), inadequate scenario adaptability, and a trade-off between real-time responsiveness and interpretability.

B. Automated Machine Learning

AutoML has emerged as a promising research field in recent years, offering a solution to the challenges associated with traditional ML methods [31], [32]. Conventional approaches heavily rely on the quality of algorithms and parameter settings, which can be challenging to optimize. AutoML enables researchers and practitioners to streamline the ML workflow by automating algorithm selection and hyperparameter tuning. This automation not only circumvents the complex task of manual hyperparameter adjustment but also enhances work efficiency for both ML experts and non-experts.

HPO [33], a key component of AutoML, automates the tuning of model hyperparameters. Its goal is to find an effective set of hyperparameters that allows the considered ML model to achieve near-optimal performance on a given dataset, thereby relieving domain experts from the laborious and error-prone task of manual tuning.

HPO methods are primarily divided into four categories: grid search, random search, sequential model-based optimization, and multiarmed bandit approaches. Grid search exhaustively evaluates every combination in the Cartesian product of hyperparameter values to identify the optimal model. Although effective, it is computationally intensive. Random search randomly samples combinations from a predefined search space, trains models with these combinations, and evaluates their performance on a validation set [34]. Although more computationally efficient than grid search, it may fail to achieve global optimality, settling on suboptimal solutions. Sequential model-based optimization uses a surrogate model (such as Gaussian processes, Tree Parzen Estimator, or random forests) along with an acquisition function. The Tree Parzen Estimator and random forest are particularly effective for handling both discrete and continuous hyperparameters [35]. Multi-armed bandit methods adaptively select hyperparameter combinations, efficiently exploring the search space and reducing computational costs compared to grid or random search [36]. These algorithms utilize prior knowledge to focus on promising regions of the hyperparameter space. Representative examples such as Auto-Sklearn [37] and Auto-WEKA [38] automate the ML pipeline—including algorithm selection and HPO—thereby enhancing the AutoML workflow.

C. AutoML-Based Anomaly Detection

AutoML-based anomaly detection techniques constitutes an intelligent detection paradigm that deeply embeds AutoML technology into the full workflow of anomaly detection. It eliminates key manual intervention procedures in traditional anomaly detection via automated methodologies, enabling the precise identification of abnormal behaviors or data features that diverge from normal patterns within target application scenarios. Simultaneously, this paradigm minimizes dependence on specialized algorithmic expertise and manual operational efforts. Chen et al. [39] developed an automated anomaly detection model utilizing statistical learning for the automatic optimization of model parameters. Yang et al. [40] proposed an AutoML-based autonomous intrusion detection system (IDS) framework designed to automatically detect malicious network attacks, thereby enhancing the security of 5G and future 6G networks. Vasan et al. [41] proposed ICS Defender, a defense mechanism that enhances the security of industrial control systems (ICS) through automated machine learning (AutoML) technology. This method employs sophisticated feature engineering and AutoML to automate model selection, training, aggregation, and optimization, thereby reducing reliance on specialized expertise. Experimental results on diverse datasets from power systems and electric vehicle charging stations demonstrate that ICS Defender outperforms existing frameworks in terms of accuracy and robustness. Ma et al. [42] proposed a few-shot IoT attack detection method based on the SSDSAE and an adaptive loss-weighted meta-residual network, which exhibits strong detection performance for scenarios with limited samples in IoT intrusion detection. Kotlar et al. [43] advanced automated anomaly detection by establishing novel meta-features for model selection, which include traditional dataset statistics and additional temporal, distributional, and domain-specific characteristics. This approach considerably improved model selection accuracy and adaptability, outperforming traditional AutoML frameworks on various public datasets—especially in managing high-dimensional sparse data and concept drift. Singh et al. [44] introduced a meta-learning-based unsupervised anomaly detection framework that creates a "data distribution–optimal detection method" by examining the distribution features of historical datasets. Experimental results indicate that this framework outperforms mainstream AutoML tools on multi-domain datasets, particularly in the detection of network traffic anomalies and industrial sensor faults in scenarios with complex and varying data distributions.

The present paper demonstrates that AutoML-based ADSs offer considerable advantages in cybersecurity. These frameworks enhance performance by swiftly adapting to evolving attack environments through automated feature engineering, intelligent model selection, and HPO. This increases detection accuracy and recall rates owing to fewer false positives and false negatives. Additionally, through the automation of tasks such as manual model design and parameter tuning, AutoML systems reduce the reliance on human experts, streamline workflows, and reduce human error and labor costs—thereby strengthening cybersecurity defenses.

III. Method

A. Problem Definition

The problem was defined as the selection of multiple algorithms with relatively optimal performance from a set of algorithms, followed by the construction of multiple models using the selected algorithms. Ultimately, the goal was to optimize the detection capabilities of the detection model. The formalized problem definition is as follows: given a set of anomaly detection algorithms $A = \{A^1, A^2, \ldots, A^n\}$ and dataset D, select algorithms $A^* \in A$ with the optimal detection capabilities from the algorithm set A. Divide the dataset into training dataset D_{train} and test dataset D_{test}. Apply algorithm A^* to obtain model M^* and then evaluate the model M* based

on the dataset D_{test}, as shown in Eq. (1).

$$A^* \in \arg\min_{A^* \in A} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(A, D_{train}, D_{test}\right) \tag{1}$$

where, $\mathcal{L}\left(A, D_{train}, D_{test}\right)$ is the loss of the model. It can be obtained by training algorithm A on dataset D_{train} and evaluating the trained algorithm on dataset D_{test}. Problem of HPO. Suppose algorithm A contains n hyperparameters, $\lambda_1, \lambda_2, \ldots, \lambda_n$, with corresponding value sets $\Lambda_1, \Lambda_2, \ldots, \Lambda_n$. Then, the hyperparameter search space Λ of algorithm A is the set formed by the Cartesian product of the value sets of these n hyperparameters: $\Lambda \subset \Lambda_1 \times \Lambda_2 \times \ldots, \times \Lambda_n$. The main purpose of HPO is to search within the search space Λ for a set of configurations λ^* that result in the optimal predictive performance of trained algorithm A, as shown in Eq. (2).

$$\lambda^* \in \arg\min_{\lambda \in \Lambda} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(A_\lambda, D_{train}, D_{test}\right) \tag{2}$$

This work mainly addresses the problem of combining algorithm selection and HPO. The problem can be defined as follows: given a set of algorithms $A = \left\{A^1, A^2, \ldots, A^n\right\}$, where the hyperparameter space corresponding to each algorithm is $\Lambda_1, \Lambda_2, \ldots, \Lambda_n$, find algorithm A^* with configuration λ^*, such that the model trained using algorithm $A^*_{\lambda^*}$ has the optimal detection performance. Accuracy, Precision, Recall, and F1-score were used as performance metrics for the anomaly detection model, as shown in Eq. (3).

$$A^*_{\lambda^*} = \arg\min_{A^j \in A, \lambda \in \Lambda_j} \frac{1}{n} \sum_{\mathrm{i}=1}^{n} \mathcal{L}\left(A^j_\lambda, D_{train}, D_{test}\right) \tag{3}$$

B. Basic Framework of the Auto-EAD

The aim of the present work was to develop an autonomous ADS for the CEN-IoT to ensure its secure operation. Fig 2 outlines the comprehensive framework of the tree-based automated ensemble anomaly detection model for the IoT (Auto-EAD). It comprises five stages: automated data preprocessing (AutoDP), automated feature engineering (AutoFE), automated base model learning and HPO, automated model selection, and automated model ensemble. In the AutoDP phase, network traffic data undergoes preprocessing steps such as transformation, imputation, normalization, and balancing to enhance the quality of training data. In the AutoFE phase, feature importance for classification tasks and the strength of the linear relationship between feature variables are computed to reduce data complexity. In the automated base model learning and HPO phase, five tree-based ML models are utilized, with HPO performed using adaptive evolution. After HPO, three detection models are selected in the automated model selection stage based on their F1-scores. The proposed Stair ensemble model is then employed in the automated model ensemble stage to stack and iteratively optimize the three base models, culminating in the final integrated ADS for the IoT. Further details on the Auto-EAD framework are presented in Algorithm 1.

Algorithm 1 Overall Tree-based Auto-EAD Framework

Input: IoT based on consumer electronics network traffic data D, feature importance threshold $\tau = 0.9$, correlation threshold $\delta = 0.9$, imbalance threshold $imbalance\ ratio = 0.3$

Output: Final Auto-EAD model

// AutoDP

$D_{clean} \leftarrow AutoDP\,(D)$ Auto data transformation: encode non-numeric features, auto imputation: fill missing values with 0, auto normalization: Z-score if normal (Shapiro–Wilk test), else min–max, auto balancing: SMOTE if $imbalance\ ratio < 0.3$ //$imbalance\ ratio$ denotes the proportion of the minority class in the dataset.

// AutoFE

$D_{reduced} \leftarrow AutoDP\,(D_{clean})$

Step 1: Remove irrelevant features via GBDT feature importance (keep those with $feature\ importance > \tau$)

Step 2: Remove redundant features via Pearson correlation $\left(\left|r_{pq}\right| > \delta\right)$

// Automated base model learning & HPO

base_models ← {Random Forest, XGBoost, Gradient Boosting, Decision Tree, LightGBM}

For each model in base_models do

optimized_model ← CMA-ES($model$, $D_{reduced}$)

end for

// Automated model selection

top_models ← select top 3 models by F1-score: $\left\{Model_{TOP\_1}, Model_{TOP\_2}, Model_{TOP\_3}\right\}$

// Automated model ensemble (Stair ensemble)

final_model ← StairStacking(top_models)

return final_model

C. Automated Data Pre-Processing

Data preprocessing is essential for automated ML, playing a crucial role in optimizing input data quality and improving the efficacy of anomaly detection models. This phase is resource-intensive, time-consuming, and requires specialized knowledge. Herein, AutoDP was employed to facilitate the learning of valuable patterns by anomaly detection models. Within the framework outlined in the present study, the AutoDP phase encompasses automatic data transformation, imputation, standardization, and balancing procedures [18]. The detailed operations are as follows:

(1) Automatic Data Transformation: Non-numeric attributes are converted into numeric ones using automated encoding techniques.

(2) Automatic Data Imputation: Missing values in the dataset are identified and replaced with zeros.

(3) Automatic Standardization: Data normality is evaluated using the Shapiro–Wilk test. If the data conforms to a normal distribution, Z-score normalization is employed; otherwise, min–max normalization is utilized.

Sample data $X_1, X_2, \ldots, X_n$ becomes $X_{(1)} \leq X_{(2)}, \ldots, X_{(n)}$ after sorting. The Shapiro–Wilk test statistic W is defined as:

$$W = \frac{\left(\sum_{i=1}^{k} a_i X_{(n-i+1)} - \sum_{i=k+1}^{n-k} a_i X_{(i)}\right)^2}{\sum_{i=1}^{n}\left(X_i - \frac{1}{n}\sum_{i=1}^{n} X_i\right)^2} \tag{4}$$

where $k = \lfloor n/2 \rfloor$ denotes the sample half-length after floor division; a_i are coefficients determined by sample size n,

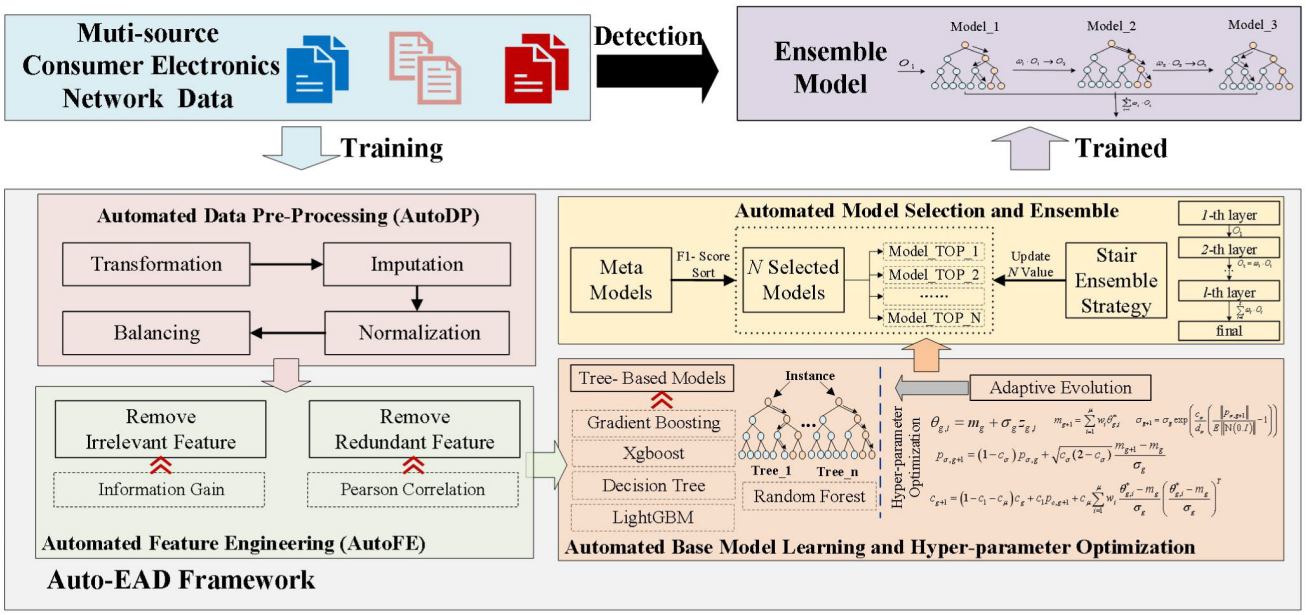

Fig. 2. Flowchart of the proposed Basic Framework of the Auto-EAD.

generated via lookup tables or algorithmic computation to reflect the covariance structure of order statistics under the normal distribution.

After computing the Shapiro–Wilk test statistic W, the critical value W_α is obtained using the Shapiro–Wilk distribution table with a significance level α ($\alpha = 0.05$) and sample size n.

In Z-score normalization, the normalized value of each data point, x_n, is denoted by

$$x_n = \frac{x - \mu}{\sigma} \tag{5}$$

where x is the original value, μ and σ are the mean and standard deviation of the data points, respectively.

In min–max normalization, the normalized value of each data point, x_n is denoted by

$$x_n = \frac{x - min}{max - min} \tag{6}$$

where x is the original feature value, and min and max are the minimum and maximum values of each original feature, respectively.

(4) Automatic Data Balancing: The class sample sizes are assessed to gauge dataset imbalance. If the dataset is imbalanced, SMOTE oversampling is applied to address such imbalance.

For each minority class sample x_j, its k-nearest neighbors in the minority class sample are calculated.

$$dist\left(x_j, x_l\right) = \sqrt{\sum_{t=1}^{d}\left(x_{j,t} - x_{l,t}\right)^2} \tag{7}$$

where $x_{j,t}$ is the t-th dimensional feature of x_j.

Thus, the set of neighboring samples $N_k\left(x_j\right) = \left\{x_{j1}, x_{j2}, .., x_{jk}\right\}$ is obtained.

Sample x_{jr} ($r \in \{1, 2, \ldots, k\}$) is randomly selected from the neighbor set $N_k\left(x_j\right)$, after which a new sample x_{new} is generated through linear interpolation.

$$x_{new} = x_j + \lambda\left(x_{jr} - x_j\right) \tag{8}$$

Among them, $\lambda \in (0, 1)$ is a randomly generated interpolation coefficient. This way, by leveraging the distribution information on the existing samples from the minority class, new minority class samples are synthesized. Consequently, the number of minority class samples is increased to alleviate the dataset imbalance.

D. AutoFE

AutoFE is another essential component of the Auto-EAD framework. Feature engineering is integral to the extraction and selection of the most pertinent features from a dataset, thereby enhancing the efficacy of ML models. AutoFE aims to streamline the conventional feature engineering workflow, reducing human involvement in such tasks. Within this framework, AutoFE focuses on feature selection, aiming to identify and select the most crucial features to construct accurate and efficient ML models.

1. Information Gain-based Irrelevant Feature Removal Method

The dataset comprises n samples, each consisting of m features and a label variable. The dataset can be represented as a feature matrix x, $x \in \mathrm{R}^{n \times m}$, and a label vector y, $y \in \mathrm{R}^n$, where x_{ij} denotes the j-th feature value of the i-th sample, and y_i denotes the label value of the i-th sample.

Herein, feature importance was assessed using the Gradient Boosting Decision Tree (GBDT) algorithm [45]. In GBDT, each decision tree is designed to model the residuals of the preceding tree. Throughout the construction of a decision tree, features are iteratively partitioned to minimize a specified loss function (e.g., mean squared error). At each decision tree node, a feature and a split point are chosen to maximize the impurity reduction of the resulting subnodes. Feature importance is quantified by either the frequency of a feature being utilized for splitting across all decision trees or the cumulative impurity reductions achieved by these splits.

Consider a set of decision trees T. The importance score of the j-th feature in the t-th decision tree can be represented as I_{tj}. The cumulative importance score of the j-th feature across the entire model is calculated by summing the importance scores across all decision trees:

$$I_j = \sum_{t=1}^{T} I_{tj} \tag{9}$$

The features are arranged in descending order based on their importance scores I_j to derive a prioritized sequence of features. These feature importance scores are normalized such that the total sum across all features is 1. The normalized importance score $\tilde{I}_j$ for the j-th feature can be mathematically defined as:

$$\tilde{I}_j = \frac{I_j}{\sum_{k=1}^{m} I_k} \tag{10}$$

Cumulative importance score C_j are calculated for each feature, which is the sum of the normalized importance scores for the top j features:

$$C_j = \sum_{k=1}^{j} \tilde{I}_k \tag{11}$$

An importance threshold τ ($\tau = 0.9$ in this paper) is defined. Features with cumulative importance scores less than or equal to the threshold τ are identified, and features with scores exceeding it are eliminated.

2. Pearson Correlation Coefficient-based Redundant Feature Removal Method

Pearson correlation coefficient ρ_{XY} assesses the strength of linear correlation between two variables. For two random variables X and Y, it is computed as follows [46]:

$$\rho_{XY} = \frac{cov(X, Y)}{\rho_X \rho_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sqrt{E\left[(X - \mu_X)^2\right] E\left[(Y - \mu_Y)^2\right]}} \tag{12}$$

where $\mathrm{cov}(X, Y)$ is the covariance of X and Y; ρ_X and ρ_Y are the standard deviations of X and Y, respectively; μ_X and μ_Y are the means of X and Y, respectively; and $\mathrm{E}[\cdot]$ is the mathematical expectation.

For two feature vectors x_p and x_q in the feature matrix x, Pearson correlation coefficient r_{pq} can be computed using the following formula:

$$r_{pq} = \frac{\sum_{k=1}^{n} \left(x_{kp} - \bar{x}_p\right)\left(x_{kq} - \bar{x}_q\right)}{\sqrt{\sum_{k=1}^{n} \left(x_{kp} - \bar{x}_p\right)^2 \sum_{k=1}^{n} \left(x_{kq} - \bar{x}_q\right)^2}} \tag{13}$$

where x_{kp} and x_{kq} are the values of the k-th sample for the p-th and q-th features, and $\bar{x}_p$ and $\bar{x}_q$ are the sample means of the p-th and q-th features, respectively.

Correlation coefficient matrix $R = \left[r_{pq}\right]$ can be derived by calculating Pearson correlation coefficients between all feature pairs. Notably, $r_{pp} = 1$ (as a feature is perfectly correlated with itself), and $r_{pq} = r_{qp}$ (because the correlation coefficient is symmetrical). To streamline computations and analyses, herein, we focused on the upper triangle of the matrix, excluding diagonal elements.

This upper triangular segment can be isolated by creating an upper triangular matrix U. A correlation coefficient threshold δ is defined (set to 0.9 in the present work). For each element r_{pq} in the upper triangle, $r_{pq} > \delta$ indicates high linear correlation between the p-th and q-th features, meaning that one of these features is redundant and needs to be removed.

E. Automated Model Learning

After processing the data with AutoDP and AutoFE, supervised ML algorithms were trained to detect attack behaviors. Herein, five tree-based ML models were considered: random forest [47], XGBoost [48], gradient boosting [49], decision trees [50], and LightGBM [51]. The rationale for selecting these models is fourfold: (1) random forest, XGBoost, LightGBM, and gradient boosting are ensemble models built on decision trees, enabling them to effectively handle non-linear and high-dimensional industrial data; (2) tree-based ML algorithms support parallel computing, thereby enhancing training efficiency on IIoT datasets; (3) the stochastic nature of tree-based ML models enhances the generalization and robustness of ensemble models.

F. Hyperparameter Optimization Based on Adaptive Evolution

HPO plays a pivotal role in the effective deployment of ML models tailored to specific IoT systems or datasets. Herein, an adaptive evolution approach was employed to automate the tuning of hyperparameter combinations, thereby considerably enhancing the detection performance of the proposed model. We used the Covariance Matrix Adaptation Evolution Strategy (CMA-ES), a robust gradient-free optimization algorithm known for its efficacy [52]. Drawing inspiration from evolutionary strategies, CMA-ES emulates the process of biological evolution to iteratively approach the optimal solution.

The primary rationale for selecting CMA-ES in this paper is fourfold: (1) As a gradient-free optimization algorithm, CMA-ES does not require assumptions regarding the continuity or differentiability of the hyperparameter space, and thus perfectly adapts to the optimization requirements of mixed-type hyperparameters in tree-based ML models. (2) The task of IoT-based network attack detection confronts challenges including dynamic variations in data distribution and high-dimensional feature spaces. By adaptively adjusting the covariance matrix, CMA-ES can dynamically balance global and local search capabilities, and rapidly converge to a robust hyperparameter configuration under constrained computing resources. Compared with alternative methods such as grid search and random search, CMA-ES offers distinct efficiency advantages, making it particularly suitable for addressing the optimization requirements of large-scale IoT datasets. (3) The inherent robustness of CMA-ES enables it to effectively mitigate noise interference during the optimization process, reduce the impact of local optimal traps via its population-based evolutionary mechanism, and ensure that the optimized hyperparameter combinations can maintain stable detection performance across diverse scenarios. This attribute is critical for anomaly detection systems requiring long-term deployment in IoT environments. (4) CMA-ES enables automatic iteration and optimization without manual intervention,

Algorithm 2 CMA-ES Algorithm for HPO

Input: Initial hyperparameter vector θ_0, mean vector m_0, step size σ_0, covariance matrix C_0, max iterations T, λ, μ, weights $\mathcal{W}_i$, learning rates $c_\sigma, d_\sigma, c_c, c_1, c_\mu$

Output: Optimized hyperparameter vector θ^*

1: $g \leftarrow 0$
2: while $g < T$ do
3: Sample λ hyperparameter vectors from $\mathcal{N}\left(m_g, \sigma_g^2, C_g\right)$: $\left\{\theta_{g,1}, \theta_{g,2,..,}\theta_{g,\lambda}\right\}$
4: for each $\theta_{g,i}$ in $\left\{\theta_{g,1}, \theta_{g,2,..,}\theta_{g,\lambda}\right\}$ do
5: Train tree-based model $\theta_{g,i}$ and compute validation loss $f\left(\theta_{g,i}\right)$
6: end for
7: Select top μ vectors $\left\{\theta^*_{g,1}, \theta^*_{g,2,}, \ldots, \theta^*_{g,\mu}\right\}$ based on $f(\theta)$
8: Update mean vector: $m_{g+1} \leftarrow \sum_{i=1}^{\mu} W_i \theta^*_{g,i}$
9: Update evolution path $p_{\sigma,g+1} \leftarrow (1-c_c)\, p_{c,g} + \sqrt{c_\sigma(2-c_\sigma)}\frac{m_{g+1}-m_g}{\sigma_g}$
10: Update step size: $\sigma_{g+1} \leftarrow \sigma_g \exp\left(\frac{c_\sigma}{d_\sigma}\left(\frac{\|p_{\sigma,g+1}\|}{E\|\mathcal{N}(0,I)\|}-1\right)\right)$
11: Update cumulative evolution path: $p_{c,g+1} \leftarrow (1-c_c)\, p_{c,g} + \sqrt{c_c(2-c_c)}\frac{m_{g+1}-m_g}{\sigma_g}$
12: Update covariance matrix: $C_{g+1} = \left(1-c_1-c_\mu\right) C_g + c_1 p_{c,g+1} p_{c,g+1}^{\mathrm{T}} + c_\mu \sum_{i=1}^{\mu} \mathcal{W}_i \frac{\theta^*_{g,i}-m_g}{\sigma_g}\left(\frac{\theta^*_{g,i}-m_g}{\sigma_g}\right)^{\mathrm{T}}$
13: $g \leftarrow g+1$
14: end while
15: return $\theta^* \leftarrow best\ \theta$ from all iterations

thereby reducing the reliance on expert experience for hyperparameter tuning. This characteristic is particularly practical for the customized deployment of ML models across diverse devices and datasets in IoT scenarios, and enables rapid adaptation to the characteristics and attack patterns of different IoT systems.

In summary, CMA-ES has been widely applied in HPO and various other domains, with its implementation details provided in Algorithm 2.

Parameter Initialization: The CMA-ES algorithm samples hyperparameter vectors from a multivariate Gaussian distribution $\mathcal{N}\left(m, \sigma^2, C\right)$, where $m_0 \in \mathrm{R}^d$ is the mean vector initialized at the center of the search space or at a randomly selected initial point. Step size σ_0 regulates the range of distribution for sampling points within the search space, with its initial value determined through experience or based on the dimensions of the search space. $C_0 \in \mathrm{R}^{d\times d}$ is the covariance matrix, which is typically initialized as the identity matrix I, signifying the independence of hyperparameters at the outset.

Sampling: In every generation g, λ hyperparameter vectors $\left\{\theta_{g,1}, \theta_{g,2}, \ldots, \theta_{g,\lambda}\right\}$ are sampled from the current multivariate Gaussian distribution $\mathcal{N}\left(m_g, \sigma_g^2, C_g\right)$. Each sample is obtained using

$$\theta_{g,i} = m_g + \sigma_g z_{g,i} \tag{14}$$

where $z_{g,i} \sim \mathcal{N}\left(0, C_g\right)$ is sampled from a multivariate Gaussian distribution with a mean of 0 and covariance matrix C_g.

Assessment: Each sampled hyperparameter vector $\theta_{g,i}$ is used to train the considered tree-based ML model, and the resulting value of the loss function, $f(\theta_{g,i})$, which is computed on the validation set.

Selection: The sampled hyperparameter vectors are then arranged in ascending order based on their corresponding loss function values. The top-performing vectors μ ($\mu < \lambda$), denoted as $\left\{\theta^*_{g,1}, \theta^*_{g,2}, \ldots, \theta^*_{g,\mu}\right\}$, are selected.

Parameter Update:

(1) Mean Vector Update

The updated mean vector m_{g+1} is calculated as the weighted average of the optimal samples:

$$m_{g+1} = \sum_{i=1}^{\mu} \mathcal{W}_i \theta^*_{g,i} \tag{15}$$

where $\mathcal{W}_i$ is the weight assigned to each sample, typically satisfying $\sum_{i=1}^{\mu} \mathcal{W}_i = 1$, with weights directly proportional to the performance (loss function value) of the samples; thus, better-performing samples are assigned higher weights.

(2) Step Size Update

The step size is adjusted based on the evolution path, decreasing when approaching the optimal solution and increasing otherwise. The update is performed as follows:

$$p_{\sigma,g+1} = (1-c_\sigma)\, p_{c,g} + \sqrt{c_\sigma(2-c_\sigma)}\frac{m_{g+1}-m_g}{\sigma_g} \tag{16}$$

$$\sigma_{g+1} = \sigma_g exp\left(\frac{c_\sigma}{d_\sigma}\left(\frac{\left\|p_{\sigma,g+1}\right\|}{E\left\|\mathcal{N}(0,I)\right\|}-1\right)\right) \tag{17}$$

where c_σ is the learning rate for step-size adaptation, d_σ is the step-size damping parameter, and $p_{\sigma,g}$ is the cumulative evolution path.

(3) Covariance Matrix Update

Cumulative evolution path $\mathrm{p_c}$ is updated as follows:

$$p_{c,g+1} = (1-c_c)\, p_{c,g} + \sqrt{c_c(2-c_c)}\frac{m_{g+1}-m_g}{\sigma_g} \tag{18}$$

The covariance matrix is updated as

$$C_{g+1} = \left(1-c_1-c_\mu\right) C_g + c_1 p_{c,g+1} p_{c,g+1}^{\mathrm{T}} + c_\mu \sum_{i=1}^{\mu} \mathcal{W}_i \frac{\theta^*_{g,i}-m_g}{\sigma_g}\left(\frac{\theta^*_{g,i}-m_g}{\sigma_g}\right)^{\mathrm{T}} \tag{19}$$

where c_1 and c_μ are the learning rates for updating the covariance matrix.

Iteration: The iterative process proceeds until meeting termination conditions, such as reaching the maximum number of iterations (T), the step-size σ falling below a specified threshold (φ), or the change in the loss function value decreasing below a certain threshold.

G. Automated Model Selection

HPO using CMA-ES is applied to all tree-based ML models. Three models (Model_TOP_1, Model_TOP_2, Model_TOP_3) with the highest F1-scores. These models are subsequently incorporated into an ensemble framework to develop the ultimate ADS, aiming to improve the classification accuracy for both normal and abnormal data patterns. The reason for selecting these models is that (1) the F1

Algorithm 3 Stair Ensemble Algorithm

Input: Trained base models $\{\text{Model}_{TOP\_1}, \text{Model}_{TOP\_2}, \text{Model}_{TOP\_3}\}$, layers L, scoring functions $\{f_1(x), f_2(x), \ldots, f_L(x)\}$

Output: Ensemble model predictions $\hat{y}$

Initialize $O_0 \leftarrow original\ data\ x$
for $l = 1$ to L do
if $l = 1$ then
$inputs \leftarrow O_0$
else
$inputs \leftarrow O_{l-1}$
end if
for each base model $h_{l,k}$ in layer l do
$y_{l,k} \leftarrow h_{l,k}(inputs)$
end if
$O_l = \Phi_l\left(\left\{\{y_{l,1}, y_{l,2}, \ldots, y_{l,k_l}\}\right\}\right)$
end for
for each layer l do
$\omega_l(x) \leftarrow \frac{\exp(f_l(x))}{\sum_{i-1}^{L}\exp(f_i(x))}$
end for
$\hat{y} = \sum_{l=1}^{L}\omega_l(x) \cdot O_l$
return $\hat{y}$

score combines both precision and recall, which can balance the classification performance of the model on normal and abnormal samples. This is particularly important for attack detection tasks, as it requires reducing false positives (improving accuracy) and reducing false negatives (improving recall). (2) After CMA-ES hyperparameter optimization, these three models showed the best comprehensive classification ability on the test set, indicating that they have stronger adaptability and stability in feature utilization, noise suppression, and pattern recognition. (3) Selecting multiple optimal models instead of a single optimal model can integrate the advantages of different models through integrated learning, increase the coverage of diversified attack modes, reduce the generalization error and overfitting risk of a single model, and thus improve the robustness and accuracy of the overall system.

H. Automated Model Ensemble

A novel ensemble learning approach termed "Stair Ensemble" was used in the Auto-EAD framework. This approach leverages adaptive weight reallocation by integrating multiple base models in a hierarchical iterative manner to construct a robust classifier with a staircase configuration. The technique emphasizes hierarchical parameter tuning and adaptive weight reallocation to augment both the detection efficacy and robustness of the model. The operational details of the proposed automated ensemble model are delineated in Algorithm 3.

Stair ensemble restructures conventional stacking into a staircase configuration comprising multiple layers L, with each layer housing numerous base models. The outputs from preceding layers is combined using adjustable weights.

The l-th layer ($1 \le l \le L$) accommodates a set of base models $\{h_{l,1}, h_{l,2}, \ldots, h_{l,K_l}\}$, with the input for this layer being the output O_{l-1} from the previous layer $l-1$ (initially, the input is the original data x). The output of each base model within the l-th layer is as follows:

$$\hat{y}_{l,k} = h_{l,k}(O_{l-1})\ k = 1, 2, \ldots, K_l \tag{20}$$

The interlayer connection function integrates the outputs of all base models in this layer as the input for the next layer:

$$O_l = \Phi_l\left(\{\hat{y}_{l,1}, \hat{y}_{l,2}, \ldots, \hat{y}_{l,k_l}\}\right) \tag{21}$$

An adaptive weight-reallocation mechanism is utilized to dynamically adjust the weights of layers based on varying data characteristics. The contribution of the output of the l-th layer (O_l), to the final prediction is determined by the weight function $\omega_l(x)$:

$$\omega_l(x) = \frac{exp(f_l(x))}{\sum_{i=1}^{L} exp(f_i(x))} \tag{22}$$

where $f_l(x)$ is a learnable scoring function, usually implemented as a neural network.

The final detection result is obtained by integrating the outputs of all layers:

$$\hat{y} = \sum_{l=1}^{L}\omega_l(x) \cdot O_l \tag{23}$$

TABLE I
ENVIRONMENT OF EXPERIMENTAL OPERATION

Type	Configuration
Processor	8 vCPUIntel(R) Xeon(R) CPU E5-2620 v4 @ 2.10GHz
Memory	40GB
Operating System	Ubuntu 18.04.5
GPU	GTX3090Ti
Video Memory	12GB
Python	3.7

TABLE II
DETAILS OF DATASETS

Datasets	Size of Train Dataset	Size of Test Dataset	Number of Features
SWaT	757375	189344	51
WADI	13824	3456	123
IoT Network Intrusion	2388795	597199	85

IV. PERFORMANCE EVALUATION

A. Experimental Environment

We employed Python 3.7 with PyTorch 1.12. as the primary deep learning framework on high-performance computing nodes. Essential libraries comprised Scikit-Learn for ML tools and Hyperopt for HPO. The detailed experimental environment is presented in Table I.

B. Datasets

Auto-EAD was evaluated on three public datasets: SWaT [53], WADI [54] and IoT Network Intrusion [55]. The detailed information on the datasets is provided in Table II.

TABLE III
DETAILS OF AUTOFE

Dataset	Number of Feature	Feature Selection
SWaT	13	LIT101, AIT201, AIT202, AIT203, FIT201, DPIT301, LIT301, AIT402, LIT401, AIT501, AIT503, AIT504, PIT502
WADI	36	Row, Time, 1_AIT_001_PV, 1_AIT_002_PV, 1_AIT_003_PV, 1_AIT_004_PV, 1_FIT_001_PV, 1_LT_001_PV, 2_DPIT_001_PV, 2_FIC_101_CO, 2_FIC_101_PV, 2_FIC_201_PV, 2_FIC_401_CO, 2_FIC_401_PV, 2_FIC_401_SP, 2_FIC_501_PV, 2_FIC_501_SP, 2_FIT_001_PV, 2_FIT_002_PV, 2_LT_001_PV, 2_LT_002_PV, 2_MCV_301_CO, 2_MCV_601_CO, 2_P_003_SPEED, 2_PIT_001_PV, 2A_AIT_004_PV, 2B_AIT_001_PV, 2B_AIT_002_PV, 2B_AIT_003_PV, 2B_AIT_004_PV, 3_AIT_003_PV, 3_AIT_004_PV, 3_AIT_005_PV, 3_LT_001_PV, LEAK_DIFF_PRESSURE, TOTAL_CONS_REQUIRED_FLOW
IoT Network Intrusion	26	Flow_ID, Src_IP, Src_Port, Dst_IP, Dst_Port, Timestamp, Flow_Duration, TotLen_Fwd_Pkts, TotLen_Bwd_Pkts, Fwd_Pkt_Len_Min, Fwd_Pkt_Len_Std, Flow_Bytss, Flow_Pktss, Flow_IAT_Mean, Flow_IAT_Std, Flow_IAT_Min, Fwd_IAT_Tot, Fwd_IAT_Mean, Bwd_IAT_Std, Bwd_Header_Len, Bwd_Pktss, Pkt_Len_Min, Pkt_Len_Mean, Pkt_Len_Std, Init_Bwd_Win_Byts, Cat

C. Evaluation Metrics

We used Accuracy, Precision, Recall, and F1-score as performance metrics, with computation formulas given in Eq. (24)- Eq. (27).

Accuracy is the proportion of the number of correctly classified samples to the total number of samples [56]:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{24}$$

Precision is the proportion of correctly classified samples among the samples classified as positives [57]:

$$Precision = \frac{TP}{TP + FP} \tag{25}$$

Recall is the proportion of successfully identified positives among all actual positives [58]:

$$Recall = \frac{TP}{TP + FN} \tag{26}$$

The F1-score is a harmonic mean of precision and recall. It provides a balanced assessment of a model by considering both precision and recall. The calculation formula is as follows [59]:

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{27}$$

All the above indicators range from 0 to 1, with higher values indicating better model performance. In the formulas, *TP*, *FP*, *TN*, and *FN* correspond to true positives, false positives, true negatives, and false negatives.

D. Details on AutoFE

We evaluated the efficiency of the Auto-EAD framework outlined in Section III on the SWaT, WADI, and IoT Network Intrusion datasets. Key features essential for the training of the Auto-EAD framework were identified through AutoDP and AutoFE, with feature importance and correlation coefficient thresholds set to 0.9. The outcomes of the final feature selection process are detailed in Table III.

E. HPO Configuration of the Best-Performing Learning Models

After AutoFE, Tree-based ML models (Random Forest, XGBoost, Gradient Boosting, Decision Tree, LightGBM) were subjected to HPO using CMA-ES. Table IV presents the optimized hyperparameters, their corresponding search spaces, and the optimal values achieved across the three datasets.

V. EXPERIMENTAL RESULTS AND DISCUSSION

A. Results for the Base Model

Fig 3–5 present the results of a comprehensive evaluation of various ML models and their optimized versions on multiple datasets. The results on the SWaT dataset indicate the robust performance of the models, with accuracy and recall approaching or exceeding 90% before and after optimization. Notably, optimization led to a modest increase in these key indicators, suggesting a limited enhancement in the detection capabilities of the models and their ability to accurately classify data samples. Importantly, the precision and F1-score are stable, indicating that the overall comprehensive performance of the models did not considerably fluctuate despite the improvements in certain metrics. The primary reason for this observation lies in the inherent characteristics of the SWaT dataset, specifically low noise levels, low dimensionality, and a simple data distribution. The initial hyperparameters of the evaluated models are close to the optimal values, which leads to a narrow hyperparameter search space and challenges in achieving significant performance improvements at the sampling points. On the WADI dataset, model optimization markedly enhanced both accuracy and precision. Additionally, some models exhibit improved recall and F1-scores after optimization, indicating that the optimization strategy effectively improved classification performance on this dataset. On the IoT Network Intrusion dataset, optimization markedly improved all performance metrics. Notably, the decision tree classifier maintains a high level of accuracy, suggesting that optimization substantially enhanced the overall efficacy of the models in network anomaly detection. To address the characteristics of high dimensionality, strong noise, and

TABLE IV
HPO CONFIGURATION OF THE BEST-PERFORMING LEARNING MODELS

Model	Hyperparameter	Configuration Space	Optimal Value on SWaT	Optimal Value on WADI	Optimal Value on IoT Network Intrusion
Random Forest	n_estimators	[50,500]	445	438	390
	max_depth	[3,20]	18	19	18
	min_samples_split	[2,20]	7	4	5
	min_samples_leaf	[1,10]	3	1	2
	max_features	['sqrt', 'log2', None]	log2	log2	None
	bootstrap	[True, False]	FALSE	FALSE	FALSE
XGBoost	n_estimators	[50,1000]	420	318	239
	max_depth	[3,12]	9	4	8
	learning_rate	[0.01,0.3] (log scale)	0.288	0.176	0.096
	min_child_weight	[1,10]	2	2	2
	gamma	[0,1.0]	0.762	0.053	0.455
	subsample	[0.6,1.0]	0.847	0.737	0.871
	colsample_bytree	0.6~1.0	0.961	0.795	0.936
	reg_alpha	0~1.0	0.11	0.592	0.192
	reg_lambda	0~1.0	0.449	0.421	0.326
Gradient Boosting	n_estimators	50~500	303	448	241
	learning_rate	0.01~0.2 (log scale)	0.166	0.117	0.131
	max_depth	[3,10]	4	8	4
	min_samples_split	[2,20]	4	7	4
	min_samples_leaf	[1,10]	1	3	9
	subsample	[0.6,1.0]	0.791	0.718	0.825
	max_features	['sqrt', 'log2', None]	sqrt	None	None
Decision Tree	max_depth	[2,20]	19	18	17
	min_samples_split	[2,20]	14	13	12
	min_samples_leaf	[1,10]	2	1	1
	max_features	['sqrt', 'log2', None]	log2	sqrt	None
	criterion	['gini', 'entropy']	entropy	gini	gini
	splitter	['best', 'random']	best	best	best
LightGBM	n_estimators	[50,1000]	427	517	658
	max_depth	[3,12]	9	7	10
	learning_rate	[0.01,0.3] (log scale)	0.094	0.077	0.019
	num_leaves	[16,256]	136	118	238
	min_child_samples	[5,100]	47	63	94

complex distributions exhibited by the WADI dataset and the IoT network intrusion dataset, the capability of the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) to integrate global and local hyperparameter search was fully leveraged. This enabled the algorithm to rapidly identify superior hyperparameter combinations and substantially enhance model performance.

The experimental results from the three datasets (i.e., SWaT, WADI, and IoT network intrusion datasets) indicate that the CMA-ES optimization algorithm exerts a positive impact on the proposed detection models. The primary reasons for this positive impact are twofold: (1) The hyperparameters of the tree-based anomaly detection models include both continuous and discrete types, and CMA-ES can fulfill the optimization requirements of mixed-type hyperparameters; (2) When confronted with dynamic variations in data distribution and high-dimensional feature spaces within IoT environments, CMA-ES can rapidly identify the optimal hyperparameter configuration by adaptively adjusting the covariance matrix.

B. Comparison With State-of-the-Art Methods

Ranked by F1-score, the top three optimized models—LightGBM_Optimized, Random Forest_Optimized, and Gradient Boosting_Optimized were selected and integrated using Traditional Stacking [60], Confidence-based Stacking [40], Hybrid Stacking [61], and the proposed Stair ensemble methods. These resulted in the Auto-TS, Auto-CS, Auto-HS, and Auto-EAD (Ours) frameworks, respectively. The rationale for selecting traditional stacking, confidence-based stacking, and hybrid stacking as comparative methods is to comprehensively validate the advantages of the proposed staircase ensemble method in feature utilization, confidence calibration, and complex boundary processing. These comparative methods cover ensemble strategies ranging from classical to improved

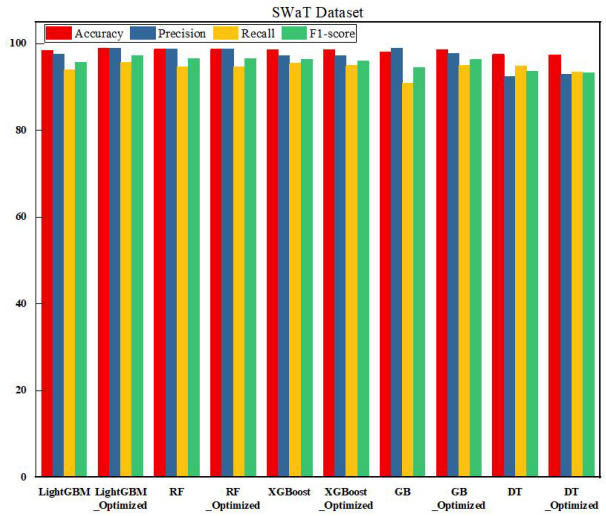

Fig. 3. Performance of the base models on the SWaT dataset.

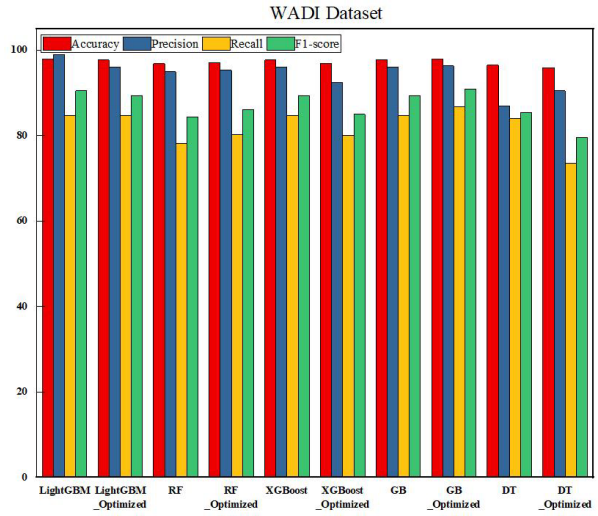

Fig. 4. Performance of the base models on the WADI dataset.

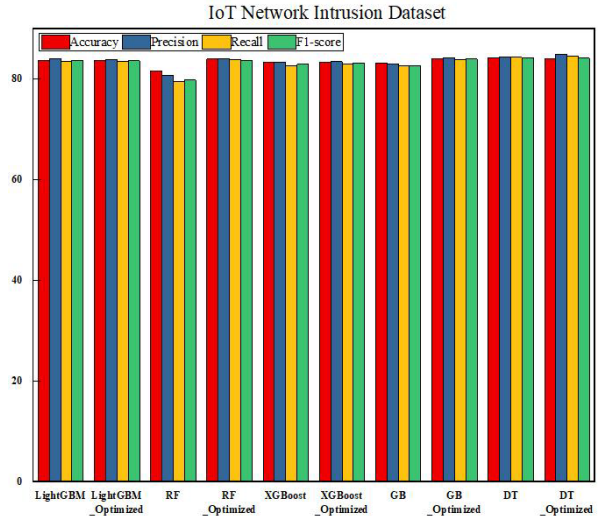

Fig. 5. Performance of the base models on the IoT network intrusion dataset.

variants, leveraging meta-learner fusion, confidence-weighted selection, and the integration of voting and stacking, respectively. This design ensures the comprehensiveness and fairness of the comparative experiments.

TABLE V
MODEL PERFORMANCE ON THE SWaT DATASET

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Auto-TS	99.000	98.933	95.683	97.236
Auto-CS	98.778	98.297	95.089	96.622
Auto-HS	98.778	98.297	95.089	96.622
CNN-FIR[62]	97.846	98.800	83.000	90.200
NN-PCA [63]	97.408	91.100	86	88.500
EPCA-HG-CNN[64]	98.020	97.710	**98.39**	**98.050**
Auto-EAD (Ours)	**99.000**	**98.933**	95.683	97.236

The performance of the proposed Auto-EAD model and several state-of-the-art methods from the literature is summarized in Table V for the SWaT dataset, Table VI for the WADI dataset, and Table VII for the IoT Network Intrusion dataset. First, as shown in Table V, Auto-EAD outperforms state-of-the-art ensemble methods and all other compared methods on the SWaT dataset, achieving 99% accuracy, 98.933% precision, 95.683% recall, and 97.236% F1-score. The Auto-EAD model exhibits exceptional performance on the WADI dataset, achieving an accuracy of 97.977%, a precision of 96.308%, a recall of 86.802%, and an F1-score of 90.926%, outperforming numerous other detection models, as presented in Table VI. At the same time, the GCN, GAT, TAGCN, FT-GCN, and OCSVM models show generally poor performance on the WADI dataset, suggesting their limited suitability for this dataset or the need for further parameter optimization. Additionally, the Auto-EAD model exhibits robust performance on the IoT Network Intrusion dataset, achieving an accuracy of 84.345%, a precision of 85.313%, a recall of 84.998%, and an F1-score of 84.651%, with a particular strength in precision. The Auto-CS model shows a maximum accuracy of 84.585%, whereas the Auto-TS and Auto-HS models exhibit comparable performance, with accuracies ranging from 84.105% to 84.185%, suggesting the stability of the integrated models. In contrast, the CNN3D model shows lower values across all indicators, with an accuracy of 82.80% and an F1-score of 81.90%, indicating the need for further performance improvement.

The proposed Auto-EAD framework outperforms the base tree-ML models and state-of-the-art models on the SWaT, WADI, and IoT Network Intrusion datasets. Such performance was primarily attributed to the synergy between AutoDP, AutoFE, HPO, and automated model ensemble modules, they collectively enhance performance by simultaneously improving data quality, optimizing ML model architectures, and reducing the complexity of features and models. In the context of model ensemble, this approach leverages adaptive weight reallocation by integrating multiple base models in a hierarchical iterative manner to construct a robust classifier with a staircase configuration. In practical IoT environments, network traffic datasets substantially surpass public benchmark datasets in complexity and variability because of the complex and

TABLE VI
MODEL PERFORMANCE ON THE WADI DATASET

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Auto-TS	97.977	96.308	86.802	90.926
Auto-CS	97.688	95.995	84.628	89.387
Auto-HS	97.688	95.995	84.628	89.387
GCN [65]	71.890	78.590	52.600	63.020
GAT [66]	73.270	78.590	54.990	65.700
TAGCN [67]	72.770	81.150	53.720	64.650
OCSVM	64.660	53.710	53.520	53.610
FT-GCN [68]	73.450	75.280	53.830	62.770
AHGA-GSAGE [69]	83.250	81.960	75.890	78.810
Auto-EAD (Ours)	**97.977**	**96.308**	**86.802**	**90.926**

TABLE VII
MODEL PERFORMANCE ON THE IOT NETWORK INTRUSION DATASET

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Auto-TS	84.105	84.184	83.855	83.968
Auto-CS	**84.585**	84.519	84.119	84.265
Auto-HS	84.185	84.191	83.814	83.957
CNN3D [70]	82.800	81.800	82.000	81.900
Auto-EAD(Ours)	84.345	**85.313**	**84.998**	**84.651**

changeable processes, the surge in the number and types of consumer electronics devices, and the deep integration with the Internet. This indicates that Auto-EAD can still be optimized and refined.

In anomaly detection, recall is of particular importance because it directly reflects the capability of a model to identify abnormal samples. A high recall rate means that the model can recognize as many actual abnormal samples as possible, which is crucial for scenarios where minimizing false negatives (missed detections) is a top priority. The proposed Auto-EAD framework achieved a remarkable 5.53% improvement in recall rate on the IoT Network Intrusion dataset. At the same time, Auto-EAD showed the highest recall rate on the WADI dataset. Furthermore, the general reliance on random selection or subjective experience in the selection of ML models often leads to suboptimal anomaly detection performance, which further highlights the advantages of the AutoML approach. This approach systematically optimizes detection models through standardized processes such as AutoDP, AutoFE, HPO, and automated model ensemble, thereby effectively enhancing detection efficiency. Therefore, the proposed AutoML-based anomaly detection framework outperforms traditional network security solutions in detection efficiency because of its autonomous design.

Overall, Auto-EAD showed superior performance in most cases, primarily because of the simplicity of the benchmark datasets and the robust performance of tree-based ML models. In real-world IoT environments, network traffic data often exhibit correlations, and data streams often show concept drift. The Auto-EAD framework, which includes components such as AutoDP, AutoFE, Automated Base Model Learning and HPO, Automated Model Selection, and Automated Model Ensemble, is likely to exhibit good performance in such scenarios. These automated components contribute to enhancing the overall anomaly detection performance, addressing the challenges faced by traditional anomaly detection models. When using traditional ML models, researchers often need to manually adjust parameters based on personal experience or changes in data streams to improve performance, which may result in suboptimal states. In contrast, the proposed ADS based on AutoML can automatically screen, optimize, and integrate the best-performing ML models, making it a promising approach with better performance than traditional network security measures, offering an autonomous solution for network security.

VI. CONCLUSION

The implementation of CEN-IoT has expanded network attack surfaces, posing notable challenges for autonomous system security. This paper introduces an Auto-EAD framework for the CEN-IoT, incorporating modules such as AutoDP, AutoFE, automated base model learning and HPO, automated model selection, and automated model ensemble. This framework aims to provide an autonomous cybersecurity solution for next-generation CEN-IoT systems. Auto-EAD exhibited superior performance on three prominent cybersecurity benchmark datasets—SWaT, WADI, and IoT Network Intrusion—achieving F1-scores of 97.236%, 90.926%, and 84.651%, respectively, and outperforming state-of-the-art anomaly detection models. This indicates the effectiveness of the proposed framework in providing autonomous network security for CEN-IoT systems or consumer electronic devices. In future studies, we plan to further optimize each module, with particular emphasis on refining the HPO algorithm. This will enhance the detection accuracy and precision, maintain computational efficiency, and address the ever-evolving nature of unknown network attack behaviors.

REFERENCES

[1] S. Qin et al., "A partially labeled anomaly data detection approach based on prioritized deep reinforcement learning for consumer electronics security," *IEEE Trans. Consum. Electron.*, vol. 70, no. 4, pp. 6452–6462, Nov. 2024.

[2] X. Li et al., "Physical-layer authentication for ambient backscatter-aided NOMA symbiotic systems," *IEEE Trans. Commun.*, vol. 71, no. 4, pp. 2288–2303, Apr. 2023.

[3] S. He et al., "Graph structure learning-based multivariate time series anomaly detection in Internet of Things for human-centric consumer applications," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 5419–5431, Aug. 2024.

[4] M. Wen et al., "A survey on spatial modulation in emerging wireless systems: Research progresses and applications," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 9, pp. 1949–1972, Sep. 2019.

[5] J. Li et al., "Composite multiple-mode orthogonal frequency division multiplexing with index modulation," *IEEE Trans. Wireless Commun.*, vol. 22, no. 6, pp. 3748–3761, Jun. 2023.

[6] W. Shang, L. Ding, X. Yang, Z. Gu, and H. Sui, "Complicated imbalanced and overlapped data oversampling approach via hypersphere coverage and adaptive differential evolution for anomaly detection of industrial Internet of Things," *IEEE Internet Things J.*, vol. 12, no. 10, pp. 14002–14015, May 2025.

[7] X. Li et al., "Hardware impaired ambient backscatter NOMA systems: Reliability and security," *IEEE Trans. Commun.*, vol. 69, no. 4, pp. 2723–2736, Apr. 2021.

[8] M. Rodríguez, D. P. Tobón, and D. Múnera, "A framework for anomaly classification in industrial Internet of Things systems," *Internet Things*, vol. 29, Jan. 2025, Art. no. 101446.

[9] L. Xiangyu, L. Tianliang, D. Yanhui, and W. Jingxiang, "Lightweight IoT intrusion detection method based on feature selection," *Netinfo Secur.*, vol. 23, no. 1, pp. 66–72, 2023.

[10] D. Javeed, M. S. Saeed, I. Ahmad, P. Kumar, A. Jolfaei, and M. Tahir, "An intelligent intrusion detection system for smart consumer electronics network," *IEEE Trans. Consum. Electron.*, vol. 69, no. 4, pp. 906–913, Nov. 2023.

[11] V. Chang et al., "A survey on intrusion detection systems for fog and cloud computing," *Future Internet*, vol. 14, no. 3, p. 89, Mar. 2022.

[12] X. Li et al., "Covert communication of STAR-RIS aided NOMA networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 6, pp. 9055–9060, Jun. 2024.

[13] M. Wen, E. Basar, Q. Li, B. Zheng, and M. Zhang, "Multiple-mode orthogonal frequency division multiplexing with index modulation," *IEEE Trans. Commun.*, vol. 65, no. 9, pp. 3892–3906, Sep. 2017.

[14] W. Kehe, C. Rui, J. Xiaochen, and Z. Jiyu, "Security protection scheme of power IoT based on SDP," *Netinfo Secur.*, vol. 22, no. 2, pp. 32–38, 2022.

[15] Y. Abudurexiti, G. Han, F. Zhang, and L. Liu, "An explainable unsupervised anomaly detection framework for industrial Internet of Things," *Comput. Secur.*, vol. 148, Jan. 2025, Art. no. 104130.

[16] M. Raeiszadeh, A. Ebrahimzadeh, R. H. Glitho, J. Eker, and R. A. F. Mini, "Real-time adaptive anomaly detection in industrial IoT environments," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 6, pp. 6839–6856, Dec. 2024.

[17] B. Ahmad, Z. Wu, Y. Huang, and S. U. Rehman, "Enhancing the security in IoT and IIoT networks: An intrusion detection scheme leveraging deep transfer learning," *Knowl.-Based Syst.*, vol. 305, Dec. 2024, Art. no. 112614.

[18] A. Liuliakov, L. Hermes, and B. Hammer, "AutoML technologies for the identification of sparse classification and outlier detection models," *Appl. Soft Comput.*, vol. 133, Jan. 2023, Art. no. 109942.

[19] Y. Li, D. Zha, P. Venugopal, N. Zou, and X. Hu, "PyODDS: An end-to-end outlier detection system with automated machine learning," in *Proc. Companion Web Conf.*, 2020, pp. 153–157.

[20] M. Bahri, F. Salutari, A. Putina, and M. Sozio, "AutoML: State of the art with a focus on anomaly detection, challenges, and research directions," *Int. J. Data Sci. Anal.*, vol. 14, no. 2, pp. 113–126, Aug. 2022.

[21] L. Zheng, D. Wu, S. Xu, and Y. Zheng, "HTCSigNet: A hybrid transformer and convolution signature network for offline signature verification," *Pattern Recognit.*, vol. 159, Mar. 2025, Art. no. 111146.

[22] Z. He et al., "A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1705–1719, Apr. 2023.

[23] W. Xiong, P. Wang, X. Sun, and J. Wang, "SiET: Spatial information enhanced transformer for multivariate time series anomaly detection," *Knowl.-Based Syst.*, vol. 296, Jul. 2024, Art. no. 111928.

[24] L. Sukhostat, "Anomaly detection in industrial control system based on the hierarchical hidden Markov model," in *Cybersecurity for Critical Infrastructure Protection via Reflection of Industrial Control Systems;*. Amsterdam, The Netherlands: IOS Press, 2022, pp. 48–55.

[25] N. Bai, X. Wang, R. Han, Q. Wang, and Z. Liu, "PAFormer: Anomaly detection of time series with parallel-attention transformer," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 2, pp. 3315–3328, Feb. 2025.

[26] Z. Gu et al., "AnomalyGPT: Detecting industrial anomalies using large vision-language models," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 3, pp. 1932–1940.

[27] C. Wu, S. Shao, C. Tunc, P. Satam, and S. Hariri, "An explainable and efficient deep learning framework for video anomaly detection," *Cluster Comput.*, vol. 25, no. 4, pp. 2715–2737, Aug. 2022, doi: 10.1007/s10586-021-03439-5.

[28] Y. Yu, X. Zeng, X. Xue, and J. Ma, "LSTM-based intrusion detection system for VANETs: A time series classification approach to false message detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23906–23918, Dec. 2022.

[29] M. S. Ansari, V. Bartoš, and B. Lee, "GRU-based deep learning approach for network intrusion alert prediction," *Future Gener. Comput. Syst.*, vol. 128, pp. 235–247, Mar. 2022.

[30] R. Sharma et al., "X hacking: The threat of misguided AutoML," 2024, *arXiv:2401.08513*.

[31] Z. Li, C. Huang, and W. Qiu, "An intrusion detection method combining variational auto-encoder and generative adversarial networks," *Comput. Netw.*, vol. 253, Apr. 2024, Art. no. 110724.

[32] K. Swathi and G. H. Bindu, "An automated intrusion detection system in IoT system using attention based deep bidirectional sparse auto encoder model," *Knowl.-Based Syst.*, vol. 305, Dec. 2024, Art. no. 112633.

[33] M. Baratchi et al., "Automated machine learning: Past, present and future," *Artif. Intell. Rev.*, vol. 57, no. 5, p. 122, Apr. 2024.

[34] C. Xue, M. Hu, X. Huang, and C.-G. Li, "Automated search space and search strategy selection for AutoML," *Pattern Recognit.*, vol. 124, Apr. 2022, Art. no. 108474.

[35] M. Wever, A. Tornede, F. Mohr, and E. Hüllermeier, "AutoML for multi-label classification: Overview and empirical evaluation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 9, pp. 3037–3054, Sep. 2021.

[36] L. Zimmer, M. Lindauer, and F. Hutter, "Auto-pytorch: Multi-fidelity metalearning for efficient and robust AutoDL," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 9, pp. 3079–3090, Sep. 2021.

[37] G. Baudart et al., "Pipeline combinators for gradual automl," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 19705–19718.

[38] M. Feurer et al., "Auto-Sklearn 2.0: Hands-free automl via meta-learning," *J. Mach. Learn. Res.*, vol. 23, no. 261, pp. 1–61, 2022.

[39] X. Chen and B. Wujek, "A unified framework for automatic distributed active learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9774–9786, Dec. 2022.

[40] L. Yang and A. Shami, "Towards autonomous cybersecurity: An intelligent AutoML framework for autonomous intrusion detection," in *Proc. Workshop Auto. Cybersecurity*, Nov. 2023, pp. 68–78.

[41] D. Vasan, E. J. S. Alqahtani, M. Hammoudeh, and A. F. Ahmed, "An AutoML-based security defender for industrial control systems," *Int. J. Crit. Infrastruct. Protection*, vol. 47, Dec. 2024, Art. no. 100718.

[42] W. Ma, L. Ma, K. Li, and J. Guo, "Few-shot IoT attack detection based on SSDSAE and adaptive loss weighted meta residual network," *Inf. Fusion*, vol. 98, Oct. 2023, Art. no. 101853.

[43] M. Kotlar, M. Punt, Z. Radivojevic, M. Cvetanovic, and V. Milutinovic, "Novel meta-features for automated machine learning model selection in anomaly detection," *IEEE Access*, vol. 9, pp. 89675–89687, 2021, doi: 10.1109/ACCESS.2021.3090936.

[44] P. Singh and J. Vanschoren, "Meta-learning for unsupervised outlier detection with optimal transport," 2022, *arXiv:2211.00372*.

[45] H. Rao et al., "Feature selection based on artificial bee colony and gradient boosting decision tree," *Appl. Soft Comput.*, vol. 74, pp. 634–642, Jan. 2019.

[46] P. Jia et al., "Erase: Benchmarking feature selection methods for deep recommender systems," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Apr. 2024, pp. 5194–5205.

[47] L. Yang, "Towards zero touch networks: Cross-layer automated security solutions for 6G wireless networks," *IEEE Trans. Commun.*, vol. 73, no. 9, pp. 7650–7679, Sep. 2025.

[48] C. Zhang, D. Jia, L. Wang, W. Wang, F. Liu, and A. Yang, "Comparative research on network intrusion detection methods based on machine learning," *Comput. Secur.*, vol. 121, Oct. 2022, Art. no. 102861.

[49] G. Mohiuddin et al., "Intrusion detection using hybridized meta-heuristic techniques with weighted XGBoost classifier," *Expert Syst. Appl.*, vol. 232, Dec. 2023, Art. no. 120596.

[50] M. H. L. Louk and B. A. Tama, "Dual-IDS: A bagging-based gradient boosting decision tree model for network anomaly intrusion detection system," *Expert Syst. Appl.*, vol. 213, Mar. 2023, Art. no. 119030.

[51] J. Liu, Y. Gao, and F. Hu, "A fast network intrusion detection system using adaptive synthetic oversampling and LightGBM," *Comput. Secur.*, vol. 106, Jul. 2021, Art. no. 102289.

[52] M. Nomura et al., "Warm starting CMA-ES for hyperparameter optimization," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 10, pp. 9188–9196.

[53] D. Zhang, "PdGAT-ID: An intrusion detection method for industrial control systems based on periodic extraction and spatiotemporal graph attention," *Comput. Secur.*, vol. 149, May 2025, Art. no. 104210.

[54] K. Wolsing et al., "Deployment challenges of industrial intrusion detection systems," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2024, pp. 453–473.

[55] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Netw. Appl.*, vol. 27, no. 1, pp. 357–370, Feb. 2022.

[56] K. Albulayhi, A. A. Smadi, F. T. Sheldon, and R. K. Abercrombie, "IoT intrusion detection taxonomy, reference architecture, and analyses," *Sensors*, vol. 21, no. 19, p. 6432, Sep. 2021.
[57] W. Aljabri, M. A. Hamid, and R. Mosli, "Lightweight and adaptive data-driven intrusion detection system for autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 2282–2292, Feb. 2025.
[58] S. Oh, S. Oh, T.-W. Um, J. Kim, and Y.-A. Jung, "Methods of pre-clustering and generating time series images for detecting anomalies in electric power usage data," *Electronics*, vol. 11, no. 20, p. 3315, Oct. 2022.
[59] A. Gorshenin, A. Kozlovskaya, S. Gorbunov, and I. Kochetkova, "Mobile network traffic analysis based on probability-informed machine learning approach," *Comput. Netw.*, vol. 247, Jun. 2024, Art. no. 110433.
[60] Z. Wang, M. Zheng, and P. X. Liu, "A novel classification method based on stacking ensemble for imbalanced problems," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–13, 2023.
[61] J. Yao, Z. Wang, L. Wang, M. Liu, H. Jiang, and Y. Chen, "Novel hybrid ensemble credit scoring model with stacking-based noise detection and weight assignment," *Expert Syst. Appl.*, vol. 198, Jul. 2022, Art. no. 116913.
[62] D. Nedeljkovic and Z. Jakovljevic, "CNN based method for the development of cyber-attacks detection algorithms in industrial control systems," *Comput. Secur.*, vol. 114, Mar. 2022, Art. no. 102585.
[63] M. Kravchik and A. Shabtai, "Efficient cyber attack detection in industrial control systems using lightweight neural networks and PCA," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2179–2197, Jul. 2022.
[64] S. P. Priyanga, K. Krithivasan, S. Pravinraj, and V. S. S. Sriram, "Detection of cyberattacks in industrial control systems using enhanced principal component analysis and hypergraph-based convolution neural network (EPCA-HG-CNN)," *IEEE Trans. Ind. Appl.*, vol. 56, no. 4, pp. 4394–4404, Jul. 2020.
[65] J. Zheng and D. Li, "GCN-TC: Combining trace graph with statistical features for network traffic classification," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2019, pp. 1–6.
[66] C. Ding, S. Sun, and J. Zhao, "MST-GAT: A multimodal spatial–temporal graph attention network for time series anomaly detection," *Inf. Fusion*, vol. 89, pp. 527–536, Jan. 2023.
[67] J. Du, S. Zhang, G. Wu, J. M. F. Moura, and S. Kar, "Topology adaptive graph convolutional networks," 2017, *arXiv:1710.10370*.
[68] X. Deng, J. Zhu, X. Pei, L. Zhang, Z. Ling, and K. Xue, "Flow topology-based graph convolutional network for intrusion detection in label-limited IoT networks," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 1, pp. 684–696, Mar. 2023.
[69] S. Lyu, K. Wang, L. Zhang, and B. Wang, "Process-oriented heterogeneous graph learning in GNN-based ICS anomalous pattern recognition," *Pattern Recognit.*, vol. 141, Sep. 2023, Art. no. 109661.
[70] I. Ullah and Q. H. Mahmoud, "Design and development of a deep learning-based model for anomaly detection in IoT networks," *IEEE Access*, vol. 9, pp. 103906–103926, 2021.

Lei Ding was born in Tianjin, China, in 1995. He is currently pursuing the Ph.D. degree in cyberspace security with the School of Cyberspace Security, Guangzhou University, Guangdong, China. His research focuses on include industrial control system information security, computational intelligence, and machine learning.

Wenli Shang was born in Heilongjiang, China, in 1974. He received the M.S. degree from the School of Mechanical and Automation Engineering, Northeastern University, in 2002, and the Ph.D. degree from the Laboratory of Industrial Control Network and System, Shenyang Institute of Automation, Chinese Academy of Sciences, in 2005. He is a Professor at Guangzhou University. His research interests include computational intelligence and machine learning, edge computing, and industrial control system information security.

Shuang Wang (Member, IEEE) was born in Heilongjiang, China, in 1986. She received the M.S. degree from the School of Computer Application Specialty, Civil Aviation University of China, in 2013, where she is currently pursuing the Ph.D. degree. She is an Assistant Researcher with the Institute of Science, Technology and Innovation, Civil Aviation University of China. Her research interests include privacy, information security, and machine learning.

Haoran Gao was born in Shaanxi, China, in 1996. He received the M.S. degree from the School of Electronic Information, Xijing University, in 2024. He is currently pursuing the Ph.D. degree with the School of Cyber Security, Guangzhou University, Guangdong, China. His research focuses on deep learning, computer vision, and backdoor attacks.

Lianhai Wang (Member, IEEE) received the M.E. degree in computer software and theory and the Ph.D. degree in computer science and technology from Shandong University, Jinan, China, in 2003 and 2014, respectively. He is currently a Research Professor with Shandong Computer Science Center (National Supercomputer Center in Jinan) and the School of Cyber Security, Qilu University of Technology (Shandong Academy of Science), Jinan. His current research interests include the areas of information security, digital forensics, and blockchain.